

**Yenepoya (Deemed to be University)**

**Project – High Level Design**

**on**

**Honeypot Network for Agriculture Threat  
Intelligence**

**Student Name:: Farhan N Methar**

**Project ID: PRJN26-205**

**Industry  
Mentor:  
SHASHANK**

**Guided by: SUZANA  
SHAROL**

## **Table of Contents**

1. Introduction.
  - 1.1. Scope of the document.
  - 1.2. Intended Audience
  - 1.3. System overview.
2. System Design.
  - 2.1. Application Design
  - 2.2. Process Flow.
  - 2.3. Information Flow.
  - 2.4. Components Design
  - 2.5. Key Design Considerations
  - 2.6. API Catalogue.
3. Data Design.
  - 3.1. Data Model
  - 3.2. Data Access Mechanism
  - 3.3. Data Retention Policies
  - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
  - 7.1. Security Aspects
  - 7.2. Performance Aspects
8. References

# 1. Introduction

## 1.1 Scope of the Document

This document provides a high-level overview of the design and architecture of the Honeypot Network for Agriculture Threat Intelligence system. It describes the overall structure, system components, data flow, and working of the system without going into low-level implementation details.

## 1.2 Intended Audience

This document is intended for project evaluators, instructors, and stakeholders who want to understand the system architecture, design decisions, and overall functionality of the honeypot system.

## 1.3 System Overview

The system is designed as a honeypot network that simulates a vulnerable AgriSense IoT Gateway in an agriculture-based environment. It listens on port 8888 and logs all incoming TCP connection attempts. The system helps in detecting suspicious activities, understanding attacker behavior, and building agriculture-specific threat intelligence.

The system listens on port 8888 and uses threshold-based logic to detect repeated connection attempts. Any IP that connects 5 or more times is flagged as Suspicious and permanently blocked. All events are logged to a CSV file and visualized using IBM Cognos Analytics.

# 2. System Design

## 2.1 Application Design

The application is designed as a server-based system using Python's Socket library. It listens for incoming TCP connections on port 8888, simulating an AgriSense IoT Gateway device. All connection attempts are recorded for monitoring, analysis, and threat intelligence purposes.

## 2.2 Process Flow

The following steps describe the complete process flow of the system:

- System is initialized and server socket is created
- Server binds to port 8888 and starts listening for connections
- Previously blocked IPs are loaded from blocked\_ips.txt into memory
- Incoming connection requests are received and accepted

- System captures IP address, port number, and timestamp of each connection
- A new thread is created to handle each connection simultaneously
- System checks if the connecting IP is already blocked
- If not blocked, connection attempt count for the IP is incremented
- System checks for repeated attempts using threshold-based detection
- If attempts reach 5 or more, IP is flagged as Suspicious and blocked
- All connection details are saved to log.csv for further analysis
- System sends a fake AgriSense IoT Gateway response to the attacker
- Connection is closed and server returns to listening mode

## 2.3 Information Flow

The following diagram shows how data flows through the system:

```

Attacker / Client
  ↓ TCP Connection on Port 8888
AgriSense Honeypot Server (honeypot.py)
  ↓ IP, Port, Timestamp extracted
Threat Detection Module (ip_count + blocked_ips)
  ↓ Connection details
CSV Logging Module (log.csv)
  ↓ log.csv data
IBM Cognos Analytics Dashboard
  ↓ Visual Reports
Threat Intelligence Output
  
```

## 2.4 Components Design

The system consists of the following major components:

| # | Component               | Description                                                                      |
|---|-------------------------|----------------------------------------------------------------------------------|
| 1 | <b>Honeypot Server</b>  | Core TCP server that listens on port 8888 and accepts all incoming connections   |
| 2 | <b>Logging Module</b>   | Records connection details (Time, IP, Port, Attempts, Status) into log.csv       |
| 3 | <b>Detection Module</b> | Identifies suspicious IPs using threshold-based logic (5+ attempts = Suspicious) |
| 4 | <b>Blocking Module</b>  | Blocks suspicious IPs and saves them permanently to blocked_ips.txt              |
| 5 | <b>Threading Module</b> | Handles multiple simultaneous connections using Python threading                 |
| 6 | <b>Analysis Module</b>  | Visualizes log data using IBM Cognos Analytics dashboard                         |

## 2.5 Key Design Considerations

- Simplicity of implementation using Python's built-in libraries only
- Real-time monitoring capability with live terminal output
- Thread-safe design to handle multiple simultaneous attackers
- Persistent IP blocking that survives server restarts
- Agriculture-specific context — simulates AgriSense IoT Gateway device
- Easy integration with IBM Cognos Analytics for visual threat reporting

## 2.6 API Catalogue

- `socket.socket()` — Creates a TCP socket for the honeypot server
- `server.bind()` — Binds the server to IP address 0.0.0.0 and port 8888
- `server.listen()` — Puts the server in listening mode
- `server.accept()` — Accepts an incoming connection and returns client and address
- `client.send()` — Sends a fake AgriSense denial response to the attacker
- `threading.Thread()` — Creates a new thread for each incoming connection
- `threading.Lock()` — Prevents race conditions in shared data access
- `csv.writer.writerow()` — Writes one connection log entry to `log.csv`

# 3. Data Design

## 3.1 Data Model

The system stores all connection data in `log.csv` with the following structure:

| Field             | Type    | Example             | Description                                |
|-------------------|---------|---------------------|--------------------------------------------|
| <b>Time</b>       | String  | 2026-04-10 15:44:01 | Date and time of the connection attempt    |
| <b>IP Address</b> | String  | 127.0.0.1           | IP address of the connecting client        |
| <b>Port</b>       | Integer | 54321               | Port used by the attacker's connection     |
| <b>Attempts</b>   | Integer | 5                   | Total connection attempts from this IP     |
| <b>Status</b>     | String  | Suspicious          | Normal if attempts < 5, Suspicious if >= 5 |

### 3.2 Data Access Mechanism

All connection data is stored in CSV format in log.csv. The file is appended to after every connection attempt. It can be opened and analyzed using Microsoft Excel, LibreOffice Calc, or IBM Cognos Analytics for dashboard visualization.

Blocked IPs are stored separately in blocked\_ips.txt. This file is read on every server startup to restore the blocked IP list into memory.

### 3.3 Data Retention Policies

Log files are stored locally and retained for the full duration of the project. There is no automatic deletion. Both log.csv and blocked\_ips.txt survive server restarts and are preserved for analysis and reporting purposes.

### 3.4 Data Migration

Not applicable. The system uses local file storage only and does not require any data migration.

## 4. Interfaces

The system uses the following interfaces:

| Interface                          | Description                                                                          |
|------------------------------------|--------------------------------------------------------------------------------------|
| <b>Command Line Interface</b>      | Used to start the honeypot server and view real-time connection logs in the terminal |
| <b>CSV File (log.csv)</b>          | Stores all connection logs for offline analysis and dashboard visualization          |
| <b>Text File (blocked_ips.txt)</b> | Stores permanently blocked IP addresses, loaded on every server restart              |
| <b>IBM Cognos Analytics</b>        | Web-based dashboard used to visualize connection data with charts and graphs         |

## 5. State and Session Management

The system maintains the following state information during execution:

- `ip_count` — A dictionary that tracks the total number of connection attempts per IP address. This is stored in memory and resets when the server restarts.
- `blocked_ips` — A set that holds all blocked IP addresses. This is loaded from `blocked_ips.txt` on startup and updated whenever a new IP is blocked.
- Each connection is handled in its own thread. A threading Lock ensures that shared state (`ip_count` and `blocked_ips`) is accessed safely without data corruption.

## 6. Caching

Caching is not implemented in this system. The `ip_count` dictionary acts as an in-memory store for connection counts during the current session, but it is not a formal caching mechanism. All persistent data is stored directly in files (`log.csv` and `blocked_ips.txt`).

## 7. Non-Functional Requirements

### 7.1 Security Aspects

- Simulates a real AgriSense IoT Gateway device to attract and trap attackers
- Detects unauthorized access attempts using threshold-based logic
- Automatically blocks suspicious IPs after 5 or more connection attempts
- Persistent IP blocking survives server restarts via `blocked_ips.txt`
- Thread-safe design prevents data corruption from simultaneous attacks
- Sends misleading fake response to confuse and deceive the attacker

### 7.2 Performance Aspects

- Lightweight system built using only Python's built-in standard libraries
- Threaded architecture allows handling multiple simultaneous connections efficiently
- CSV-based logging is fast and requires minimal system resources
- Lock mechanism adds minimal overhead while ensuring complete data integrity
- Startup loads blocked IPs instantly from file into memory for fast lookup

## 8. References

- Python Official Documentation — <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation — <https://docs.python.org/3/library/threading.html>
- Python CSV Documentation — <https://docs.python.org/3/library/csv.html>
- What is a Honeypot — <https://www.kaspersky.com/resource-center/threats/what-is-a-honeypot>
- IBM Cognos Analytics — <https://www.ibm.com/products/cognos-analytics>
- Network Security Concepts — Forouzan, B.A., Data Communications and Networking, McGraw-Hill





Innovation Centre for Education

